



SUPER AI ENGINEER SEASON 6 / DESIGN & TECHNICAL REPORT

The IoT Attack Detection Challenge

การตรวจจับการโจมตีบนเครือข่าย IoT ด้วย Protocol Rules, Stream Structure และ PCAP Evidence

PUBLIC F1

0.96267

PRIVATE F1

0.97198

สารบัญ

1. บทสรุปผู้บริหาร
2. โจทย์และเงื่อนไขการแข่งขัน
3. ข้อมูลและคุณลักษณะ
4. เหตุผลที่ไม่ใช้ Autoencoder อย่างเดียว
5. สถาปัตยกรรมแนวทาง
6. กฎตรวจจับหลัก
7. Precision corrections และ structural completion
8. Controlled leaderboard experiments
9. การตรวจสอบด้วยข้อมูลต้นทางและ PCAP
10. การจัดการ F1 และ candidate precision
11. ผลสุดท้ายและการตีความ
12. สิ่งทดลองแล้วไม่สำเร็จ
13. Reproducibility และการควบคุมคุณภาพ
14. แนวทางพัฒนาต่อบนเครื่องแรง
15. สรุป
16. ภาคผนวก A: MQTT message types ที่ใช้
17. ภาคผนวก B: Artifact สำคัญ
18. ภาคผนวก C: ข้อความนำเสนอ 60 วินาที

1. บทสรุปผู้บริหาร

โจทย์นี้ต่างจากงาน binary classification ปกติ เพราะ `X_train.csv` จำนวน 100,000 แถวมีเฉพาะข้อมูล Normal แต่ `X_test.csv` จำนวน 10,000 แถวมีทั้ง Normal และ Attack ดังนั้นเราไม่มีตัวอย่าง Attack ที่ติด label สำหรับฝึก supervised classifier โดยตรง งานจึงอยู่ในรูปแบบ one-class classification หรือ anomaly detection

แนวทางที่ใช้ไม่ได้พยายามสร้าง anomaly score เพียงค่าเดียว แต่ประกอบ detector จากความรู้ระดับโปรโตคอล ได้แก่รูปแบบ TCP/MQTT ที่ไม่เกิดในชุด Normal รูปแบบของการโจมตีทั้ง 5 กลุ่ม ความสอดคล้องภายใน TCP stream และหลักฐานจากข้อมูล PCAP ต้นทาง จากนั้นใช้ Kaggle submission อย่างจำกัดเป็น controlled experiment เพื่อยืนยันหรือหักล้างสมมติฐานที่ละกลุ่ม

ผลลัพธ์สำคัญ:

- เพิ่ม Public F1 จาก 0.54898 ในรอบแรกเป็น 0.96267 ในไฟล์สุดท้ายที่เลือก
- ไฟล์ที่เลือกได้ Private F1 0.97198
- ไฟล์ `submission_v8_micro_best.csv` ได้ Private F1 0.97232 แม้ Public F1 ต่ำกว่าเล็กน้อยที่ 0.96193 แสดงความเสี่ยงของการเลือกโมเดลจาก Public leaderboard เพียงอย่างเดียว
- สแกน attack PCAP ครบ 313/313 ไฟล์ ประมาณ 30.4 GB และ 215.6 ล้าน TCP packets
- สแกน normal PCAP ครบ 49/49 ไฟล์ ประมาณ 7.0 GB และ 45.4 ล้าน TCP packets
- การเพิ่ม `Id=8150` จาก shape match เพียงระดับ packet ทำให้ Public F1 ลดจาก 0.96230 เป็น 0.96196 จึงยกเลิกสมมติฐานทันที
- การเพิ่ม `Id=1145` ซึ่งเป็น payload pattern ที่คัดไว้อย่างเข้มงวด ทำให้ Public F1 เพิ่มจาก 0.96230 เป็น 0.96267

2. โจทย์และเงื่อนไขการแข่งขัน

2.1 เป้าหมาย

ทำนาย label ของแต่ละ record ใน `X_test.csv`:

- 0 หมายถึง Normal communication
- 1 หมายถึง Network attack

ตัวชี้วัดคือ F1-score ซึ่งสมดุลระหว่าง Precision และ Recall:

```
Precision = TP / (TP + FP)
Recall    = TP / (TP + FN)
F1        = 2 * Precision * Recall / (Precision + Recall)
```

F1 ทำให้การเพิ่ม label เป็น Attack แบบกว้างเกินไปมีความเสี่ยงสูง เพราะ False Positive จะลด Precision ขณะเดียวกัน detector ที่แคบเกินไปจะมี False Negative สูงและ Recall ต่ำ

2.2 รูปแบบ leaderboard

- Public Leaderboard ใช้ test 50% สำหรับ feedback ระหว่างพัฒนา
- Private Leaderboard ใช้ test อีก 50% สำหรับตัดสินผลสุดท้าย
- ส่งคำตอบได้ 5 ครั้งต่อวัน ตัดรอบเวลา 07:00 น. ตามเวลาประเทศไทย
- Baseline จากผู้จัดเป็น Fully-Connected Autoencoder ได้ Public 0.62303 และ Private 0.65030

2.3 ประเภทการโจมตี

ชุด test ครอบคลุมการโจมตี 5 รูปแบบ:

1. Dictionary Attack — ทดลองข้อมูลรับรองหรือรูปแบบ CONNECT ผิดปกติ
2. Invalid Subscription Attack — ส่ง SUBSCRIBE/SUBACK ที่ผิดปกติหรือปริมาณสูง
3. Dictionary Attack with Will Payload — ผสมการเดารหัสกับ MQTT Will message
4. Publish Attack — ส่ง PUBLISH payload ในรูปแบบหรือความถี่ผิดปกติ
5. TCP SYN Flooding Attack — ส่ง SYN จำนวนมากเพื่อใช้ทรัพยากรปลายทาง

3. ข้อมูลและคุณลักษณะ

3.1 ไฟล์ข้อมูล

ไฟล์	จำนวนแถว	จำนวน features	label
X_train.csv	100,000	23	Normal ทั้งหมด
X_test.csv	10,000	23	Normal + Attack ไม่เปิดเผย
sample_submission.csv	10,000	2	โครงสร้าง Id, label

3.2 กลุ่ม features

ระดับเฟรมและโปรโตคอล

- `frame.len` — ขนาด frame ทั้งหมด
- `frame.protocols` — protocol stack เช่น `eth:ethertype:ip:tcp:mqtt`

ระดับ TCP

- `tcp.stream` — หมายเลข stream ใช้ตรวจบริบทภายใน connection/capture
- `tcp.analysis.initial_rtt` — initial round-trip time
- `tcp.len` — TCP payload length
- `tcp.window_size` — advertised/calculated TCP window
- `tcp.flags.syn`, `tcp.flags.reset`, `tcp.flags.ack` — TCP state flags

ระดับ MQTT

- `mqtt.msgtype` — CONNECT, CONNACK, PUBLISH, SUBSCRIBE, SUBACK, PINGREQ, PINGRESP, DISCONNECT
- `mqtt.qos`, `mqtt.conflag.qos` — Quality of Service
- `mqtt.conflag.cleansess`, `mqtt.kalive` — session behavior
- `mqtt.username_len`, `mqtt.passwd_len` — credential-length signatures
- `mqtt.retain`, `mqtt.conflag.retain`, `mqtt.conflag.willflag` — publish/will flags
- `mqtt.topic_len`, `mqtt.len` — topic และ message length
- `mqtt.conack.val` — CONNECT result code

3.3 คุณภาพข้อมูลที่ต้องระวัง

- MQTT columns เป็น missing เมื่อ packet ไม่มี MQTT layer ซึ่งเป็น missing by design ไม่ใช่ข้อมูลเสีย

- `tcp.analysis.initial_rtt` มีค่าเฉพาะ packet/stream ที่ Wireshark คำนวณได้
- `tcp.stream` เป็นหมายเลขภายใน capture ไม่ใช่ global identity ระหว่างไฟล์
- record จาก stream เดียวกันมีความสัมพันธ์สูง การ split แบบ random row เสี่ยง data leakage
- feature ที่เป็น length และ TCP window มีความสัมพันธ์กับ operating system, network stack และ capture environment จึงเกิด domain shift ได้

4. เหตุผลที่ไม่ใช้ Autoencoder อย่างเดียว

Autoencoder เหมาะกับ normal-only training แต่ reconstruction error ไม่ได้เท่ากับ attack probability เสมอไป ปัญหาที่พบคือ packet ปกติบางชนิดมีโครงสร้างหายากและได้ anomaly score สูง ขณะที่ attack บางชนิดใช้ packet format ที่ถูกต้องแต่เปลี่ยนความถี่ ลำดับ หรือค่าบาง field เพียงเล็กน้อย

ข้อจำกัดของ anomaly score เดี่ยว:

- แยก rare-normal ออกจาก attack ได้ยาก
- ไม่ใช้ความหมายของ MQTT message type โดยตรง
- ไม่ใช้ความสัมพันธ์ใน TCP stream
- threshold ที่ดีที่สุดบน Public อาจไม่ดีที่สุดบน Private
- external data มี TCP-stack shift ทำให้ classifier เรียนรู้แหล่งข้อมูลแทนชนิดการโจมตี

จึงเปลี่ยนเป็น detector แบบ precision-first: เริ่มจากรูปแบบที่มีเหตุผลจาก protocol และไม่ปรากฏใน Normal แล้วค่อยเพิ่ม exception ที่ผ่านหลักฐานหลายชั้น

5. สถาปัตยกรรมแนวทาง

```
X_train (Normal only) —> Normal profile / known packet shapes —┐
                                                                    └-> Rule union -> Precision
corrections -> Submission
X_test —————> TCP + MQTT protocol signatures —————┐
Source CSV/PCAP —————> Exact/shape support + normal falsifier ┘
Kaggle controlled probes —————┘
```

ขั้นตอนหลัก:

1. สร้าง normal profile จากค่าที่เกิดขึ้นจริงใน `X_train`
2. สร้างกฎสำหรับ attack family ที่อธิบายได้
3. รวมผลกฎด้วย logical OR เพื่อเพิ่ม Recall
4. ตัด hard false positives ด้วย exclusion rules เพื่อรักษา Precision
5. ตรวจสอบ structural completeness ภายใน TCP stream
6. ตรวจสอบหลักฐานกับ source CSV/PCAP และ normal PCAP
7. ส่ง controlled probe ที่ละกลุ่มและใช้ score delta ตัดสินสมมติฐาน
8. validate schema, Id order, binary labels และ checksum ก่อนส่งทุกครั้ง

6. กฎตรวจจับหลัก

6.1 TCP window ที่ไม่เคยปรากฏใน Normal

```
attack_stack = ~test["tcp.window_size"].isin(normal_windows)
```

เหตุผล: TCP window สะท้อน network stack และ attack generator บางชุดใช้ค่าเฉพาะที่ไม่เกิดใน normal devices อย่างไรก็ตามกฎนี้ต้องใช้ร่วมกับกฎอื่นเพราะ window shift อาจเกิดจากอุปกรณ์ใหม่ได้

6.2 คู่ frame length และ window ที่ไม่เคยปรากฏใน Normal

```
attack_packet_shape = (  
    ~test_frame_windows.isin(normal_frame_windows)  
    & test["mqtt.msgtype"].ne(3)  
)
```

ตัด PUBLISH ออกจากกฎ novelty นี้ เพราะ PUBLISH มีความหลากหลายของ payload length สูง หากใช้ novelty ตรงๆ จะเกิด false positive มาก

6.3 TCP SYN flooding

```
syn_flood = (  
    test["frame.len"].eq(54)  
    & test["tcp.window_size"].eq(512)  
    & test["tcp.flags.syn"].eq(1)  
    & test["tcp.flags.ack"].eq(0)  
)
```

รูปแบบคือ SYN packet ขนาด 54 bytes ไม่มี ACK และใช้ window 512 สอดคล้องกับ SYN flood generator ในชุดข้อมูล

6.4 Dictionary CONNECT

```
dictionary_connect = (  
    test["mqtt.msgtype"].eq(1)  
    & test["mqtt.kalive"].isin([60, 3600])  
)
```

CONNECT packets เป็นจุดที่เห็น credential/session configuration การโจมตีแบบ dictionary ในชุดนี้มี keep-alive signature ที่แยกจาก normal profile

6.5 Invalid subscription

```
invalid_subscription = test["mqtt.msgtype"].isin([8, 9])
```

Message type 8 และ 9 คือ SUBSCRIBE/SUBACK ซึ่งสัมพันธ์กับ invalid subscription captures ในข้อมูลการแข่งขัน

6.6 Publish attack

```
publish_attack = (  
    test["mqtt.msgtype"].eq(3)  
    & test["mqtt.len"].isin([19, 44])  
)
```

PUBLISH attack ใช้ payload signature เฉพาะ จากนั้น controlled probe ยืนยัน capture family เพิ่มเติมที่ `mqtt.msgtype=3` และ `tcp.window_size=256`

7. Precision corrections และ structural completion

7.1 ตัด PINGRESP hard negative

Pseudo-label model เคยให้ attack probability สูงกับ PINGRESP บางกลุ่ม แต่ submission probe แสดงว่าเป็น false positive จึงเพิ่ม exclusion:

```
pingresp_false_positive = (  
    test["frame.len"].eq(56)  
    & test["tcp.window_size"].eq(253)  
    & test["mqtt.msgtype"].eq(13)  
)  
labels &= ~pingresp_false_positive
```

บทเรียน: model confidence สูงไม่เพียงพอ ต้องผ่าน known-negative check

7.2 ปิดช่องว่าง SYN stream ด้วย Id=9816

การตรวจ stream พบ SYN-flood positives ครอบคลุม 599 streams ในช่วง 0-599 และขาด stream 194 เพียง stream เดียว ภายใน stream 194 มี normal template 9 packets และมี packet ส่วนเกินเพียง Id=9816 จึงเพิ่มแถวนี้แบบ structural completion

ผลจริง:

- ก่อนเพิ่ม: Public 0.96193
- หลังเพิ่ม: Public 0.96230

7.3 เพิ่ม payload twin ด้วย Id=1145

Residual ranking พบ TCP payload รูปแบบ 189-byte frame, 135-byte TCP payload, window 5738 ใน stream ที่มี attack fraction สูง การเพิ่ม Id=1145 ทำให้ Public F1 ดีขึ้นอีก:

- ก่อนเพิ่ม: Public 0.96230
- หลังเพิ่ม: Public 0.96267

Id=5244 มีทุก feature นอกจาก Id เหมือน Id=1145 จึงถูกเก็บเป็น candidate ถัดไป แต่ไม่ถูกส่งเพราะ quota หมด

8. Controlled leaderboard experiments

การใช้ leaderboard มีเป้าหมายเป็น hypothesis test ไม่ใช่ brute-force label extraction:

- 1. เปลี่ยน label เฉพาะกลุ่มที่มีเหตุผล
- 2. บันทึก changed Id list และ checksum
- 3. เทียบ score delta กับ control
- 4. เก็บเฉพาะกฎที่ดีขึ้นอย่างสม่ำเสมอ
- 5. ยกเลิกกฎทันทีเมื่อ falsified

Submission	Public F1	Private F1	ข้อสรุป
submission_first.csv	0.54898	0.56584	detector เริ่มต้น recall/precision ไม่สมดุล
submission_improved.csv	0.93152	0.93907	protocol rules เพิ่มผลอย่างมาก
submission_pair_novelty.csv	0.91143	0.92637	novelty กว้างเกินไป ทำให้ FP สูง
submission_control_novelty.csv	0.95649	0.96648	rollback กลุ่มเสี่ยง พื้นคะแนน
submission_final_probe.csv	0.95834	0.96856	PUBLISH family เพิ่ม recall
validated PUBLISH completion	0.96193	—	precision-first champion เดิม
structural Id=9816	0.96230	0.97198	สมมติฐาน stream completion สำเร็จ
shape Id=8150	0.96196	0.97198	สมมติฐาน packet shape ถูกยกเลิก
v8_micro_best	0.96193	0.97232	Private ดีกว่า แม้ Public ต่ำกว่า
payload Id=1145	0.96267	0.97198	Public ดีที่สุดของไฟล์ที่ส่ง

9. การตรวจสอบด้วยข้อมูลต้นทางและ PCAP

9.1 Exact source matching

ตรวจ external CSV captures ด้วย key หลัก: frame length, TCP stream, RTT ที่บิต 6 ตำแหน่ง, TCP length, window, SYN, RST และ ACK

- source rows ที่อ่าน: 2,385,747
- exact test union: 229 แถว
- ทั้ง 229 แถวถูก label เป็น Attack อยู่แล้ว
- candidate additions: 0

ผลนี้สนับสนุน precision ของกฎเดิม แต่ไม่สร้าง residual candidate ใหม่

9.2 Exhaustive PCAP scan

Corpus	ไฟล์สำเร็จ	ขนาดรวม	TCP packets
Attack PCAP	313/313	30.4 GB	215.6 ล้าน
Normal PCAP	49/49	7.0 GB	45.4 ล้าน

ใช้ profile hash หลายระดับ:

- `tcp_shape` — TCP/frame fields หลัก
- `packet` — เพิ่ม protocol และ MQTT packet fields
- `packet_stream` — เพิ่ม stream context
- `packet_rtt` — เพิ่ม initial RTT
- `packet_full` — feature ครบตามการแข่งขัน

หลักการสำคัญคือ normal falsification: candidate ต้องมี attack support และไม่ชน normal support ใน profile ที่เหมาะสม อย่างไรก็ตาม `Id=8150` ผ่านเพียง packet-shape support จาก attack PCAP แต่ไม่มี full/RTT support เมื่อส่งจริง score ลดลง จึงพิสูจน์ว่าความเหมือนระดับ packet เดี่ยวไม่เพียงพอ

10. การจัดการ F1 และ candidate precision

เมื่อ baseline มี F1 สูง การเพิ่ม positive prediction จะช่วยก็ต่อเมื่อ precision ของกลุ่มใหม่สูงพอ จาก confusion-state ที่ประมาณไว้ กลุ่ม residual ต้องมี precision ราว 0.486 หรือสูงกว่าเพื่อคาดหวัง F1 ที่ดีขึ้น

ผลต่อการตัดสินใจ:

- ไม่เพิ่ม candidate จำนวนมากเพียงเพราะ anomaly score สูง
- เลือก single-row หรือ homogeneous family ก่อน
- ประเมิน worst-case และ Monte Carlo sensitivity
- รักษา scored baseline เป็น immutable artifact
- สร้าง candidate ใหม่เป็น delta จาก baseline ไม่เขียนทับไฟล์เดิม

การจำลอง 20,000 รอบจัด structural file เป็นอันดับหนึ่งในชุด offline และชนะ 15/15 conservative stress scenarios แต่รายงานยังระบุว่า offline proxy ไม่ใช่คะแนน Kaggle รับประกัน

11. ผลสุดท้ายและการตีความ

11.1 คะแนน

- Baseline ผู้จัด: Public 0.62303, Private 0.65030
- ไฟล์ที่เลือกของโครงการ: Public 0.96267, Private 0.97198
- Private leaderboard อันดับสูงสุด: 0.98149
- อันดับสอง: 0.97879
- กลุ่มคะแนนถัดมา: 0.97198 หลายทีม รวมทีม 610686-วชิรวิทย์

คะแนน Private สูงกว่า Public ประมาณ 0.00931 แสดงว่า detector generalize ไปยัง private split ได้ดี แม้ยังมีช่องว่างจากอันดับหนึ่งประมาณ 0.00951

11.2 Public–Private selection gap

ไฟล์ `v8_micro_best` ได้ Private 0.97232 สูงกว่าไฟล์ที่เลือก 0.00034 แต่ Public ต่ำกว่า 0.00074 เหตุการณ์นี้ยืนยันว่า Public 50% มี sampling variance การเลือกโมเดลควรใช้:

- protocol evidence
- offline stress tests
- stability across candidate families
- Public score เป็นเพียงหนึ่งสัญญาณ

ไม่ควรเลือกจาก Public สูงสุดอย่างเดียว

12. สิ่งทดลองแล้วไม่สำเร็จ

12.1 Broad novelty union

เพิ่ม packet ที่ไม่เคยเห็นใน Normal จำนวนมาก ทำให้ Public ลดถึง 0.91143 สาเหตุคือ rare-normal ถูกจัดเป็น Attack

12.2 Semi-supervised pseudo labels

Extra Trees/Isolation-style residual ranking ให้คะแนนสูงกับ PINGRESP hard negatives จึงไม่ผ่าน falsification gate และถูกปฏิเสธ

12.3 External attack classifier

โมเดลแยก external attacks จาก competition normal ได้ง่ายเกินจริง เพราะเรียนรู้ TCP stack/capture source เมื่อย้ายกลับมา test domain ความน่าเชื่อถือลดลง

12.4 Packet-shape support เพียงชั้นเดียว

Id=8150 มี attack packet-shape support แต่ไม่มีหลักฐาน full context การส่งจริงทำให้คะแนนลด จึงต้องใช้ flow/transaction context ในงานต่อไป

13. Reproducibility และการควบคุมคุณภาพ

13.1 สร้าง submission หลัก

```
python outputs/predict_final_model.py \  
--train data/X_train.csv \  
--test data/X_test.csv \  
--output outputs/submission_reproduced.csv
```

ตัว predictor หลักสร้าง 2,809 positive labels จาก protocol rules จากนั้น late-stage structural/payload additions ถูกเก็บเป็น immutable submission artifacts แยกต่างหาก

13.2 Validation ก่อนส่ง

- จำนวนแถวเท่ากับ 10,000
- `Id` ไม่ซ้ำและเรียงตรง `X_test.csv`
- columns เท่ากับ `Id, label`
- label มีเพียง 0 และ 1
- บันทึกจำนวน positive labels
- บันทึก changed Id เทียบ baseline
- บันทึก SHA-256 checksum
- เก็บ submission message และ Kaggle reference

ไฟล์ที่เลือก:

```
outputs/submission_current_best_96267.csv  
rows          = 10,000  
positives     = 2,811  
sha256        = 2fe54d9dd8fc524d605fb0115941e896c86d09a6be5485d1ab626f66375c03a4  
Kaggle ref    = 54808193
```

14. แนวทางพัฒนาต่อบนเครื่องแรง

14.1 เปลี่ยนหน่วยวิเคราะห์จาก packet เป็น flow

สร้าง features จาก packet ก่อนหน้า/ถัดไปใน stream:

- inter-arrival time
- packet direction
- rolling packet count และ byte count
- TCP state transition
- SYN/ACK ratio
- MQTT transaction sequence เช่น CONNECT → CONNACK → PUBLISH
- burstiness และ repetition rate

14.2 Group-held-out validation

แบ่ง fold ตาม capture file หรือ session ไม่แบ่งแบบ random row เพื่อลด leakage จาก packet ที่อยู่ติดกัน ควรรายงาน mean, standard deviation และ worst-fold F1

14.3 Model candidates

- CatBoost/LightGBM บน packet + flow aggregates
- one-class model สำหรับ normal profile แต่ calibrate ด้วย hard negatives
- sequence model ขนาดเล็กบน event tokens
- rule-model ensemble โดยให้ protocol rules เป็น high-precision anchors

14.4 Candidate gate

candidate ใหม่ควรผ่านทุกเงื่อนไข:

1. หลักฐาน attack capture
2. ไม่ชน normal capture ใน profile ที่เหมาะสม
3. ผ่าน capture-held-out folds
4. ไม่ชน known hard negatives
5. expected precision สูงกว่า F1 break-even threshold
6. changed Id list มีขนาดเล็กและตรวจจอธิบายได้

15. สรุป

หัวใจของโครงการไม่ใช่การใช้โมเดลซับซ้อนที่สุด แต่คือการเปลี่ยนข้อจำกัด "มีแต่ Normal labels" ให้เป็นกระบวนการตรวจจับที่อธิบายได้และควบคุมความเสี่ยงได้ กฎจาก MQTT/TCP ให้ high-precision anchors, structural analysis ช่วยปิดช่องว่าง, PCAP audit ใช้ยืนยันและหักล้างสมมติฐาน และ controlled probes แปลง submission quota ให้เป็นการทดลองที่มีข้อมูล

ผล Public 0.96267 และ Private 0.97198 สูงกว่า baseline อย่างมาก ขณะเดียวกันผล v8_micro_best ที่ Private สูงกว่าไฟล์ Public-best ย้ำบทเรียนสำคัญ: งาน anomaly detection ต้องวัดความเสถียรของเหตุผลและ distribution shift ไม่ใช่ไล่คะแนน Public เพียงอย่างเดียว

ภาคผนวก A: MQTT message types ที่ใช้

ค่า	ชื่อ	ความหมาย
1	CONNECT	ขอเชื่อมต่อ broker
2	CONNACK	ตอบรับการเชื่อมต่อ
3	PUBLISH	ส่ง message ไปยัง topic
4	PUBACK	ยืนยัน PUBLISH QoS 1
5	PUBREC	ขั้นตอน QoS 2
6	PUBREL	ขั้นตอน QoS 2
7	PUBCOMP	จบ QoS 2
8	SUBSCRIBE	ขอ subscribe topic
9	SUBACK	ตอบรับ subscription
12	PINGREQ	ตรวจสอบสถานะ connection
13	PINGRESP	ตอบ PINGREQ
14	DISCONNECT	ยุติ connection

ภาคผนวก B: Artifact สำคัญ

Artifact	หน้าที่
outputs/predict_final_model.py	deterministic rule predictor
outputs/submission_current_best_96267.csv	ไฟล์ที่ได้ Public 0.96267
outputs/submission_next_01_twin_payload132.csv	candidate ถัดไปจาก exact feature twin
outputs/offline_benchmark_report.md	Monte Carlo และ sensitivity analysis
outputs/pcap_attack_shape_support.csv	attack PCAP profile support
outputs/pcap_normal_shape_support.csv	normal falsification support
outputs/strong_machine_handoff.md	handoff สำหรับเครื่องใหม่
work/search_pcap_shape_support.py	resumable multi-profile PCAP scanner

ภาคผนวก C: ข้อความนำเสนอ 60 วินาที

"โจทย์นี้มีข้อมูลฝึกที่เป็น Normal ทั้งหมด จึงไม่ใช่ classification แบบทั่วไป ผมเริ่มจากสร้าง normal profile แล้วใช้ความหมายของ TCP และ MQTT ระบุรูปแบบโจมตีที่อธิบายได้ จากนั้นตัด false positives ด้วย controlled probes ตรวจสอบความครบถ้วนระดับ stream และสแกน PCAP ต้นทางกว่า 37 GB เพื่อยืนยันหรือหักล้าง candidate ผลคือคะแนนเพิ่มจาก Public 0.54898 เป็น 0.96267 และ Private 0.97198 บทเรียนสำคัญคือ packet anomaly เพียงอย่างเดียวไม่พอ ต้องใช้ protocol context, flow structure และการเลือกโมเดลที่ไม่ overfit Public leaderboard"

อ้างอิง: เอกสารโจทย์ SuperAI6, Kaggle submission history และ artifacts ใน repository